



LV13 std::vector 클래스 만들기 (Implementing std::vector)

소유자	암암코딩
태그	
생성 일시	2024년 9월 8일 오후 12:49

https://youtu.be/H2288e_kM8k?feature=shared

```
#include <iostream>

namespace ya
{
    template <typename T>
    class Vector
    {
    public:
        // 기본 생성자
        Vector(size_t cap = DEFAULT_CAP)
            : mArr(new T[cap])
            , mSize(0)
            , mCapacity(cap)
        {
        }

        // 복사 생성자(깊은 복사)
        Vector(const Vector& other)
            : mArr(new T[other.mCapacity])
            , mSize(other.mSize)
            , mCapacity(other.mCapacity)
        {
        }
    };
}
```

```

    {
        for (size_t i = 0; i < mSize; i++)
            mArr[i] = other[i];
    }

    // 소멸자
    ~Vector()
    {
        delete[] mArr;
    }

    // 복사 대입 연산자
    Vector& operator=(const Vector& other)
    {
        if (this != &other)
        {
            if (mCapacity < other.mCapacity)
            {
                delete[] mArr;
                mCapacity = other.mCapacity;
                mArr = new T[mCapacity];
            }

            mSize = other.mSize;
            for (size_t i = 0; i < mSize; i++)
                mArr[i] = other.mArr[i];
        }
        return *this;
    }

    // 원소 접근
    T& operator[](size_t idx) { return mArr[idx]; }
    const T& operator[](size_t idx) const { return mArr
[idx]; }

    T& front() { return mArr[0]; }
    T& back() { return mArr[mSize - 1]; }

    // 반복자(포인터 기반)

```

```

T* begin() const { return mArr; }
T* end() const { return mArr + mSize; }

// 요소 추가/삭제
void push_back(const T& value)
{
    if (mSize >= mCapacity)
    {
        size_t newCap = mCapacity < DEFAULT_CAP ? D
EFAULT_CAP : mCapacity * 2;
        T* newArr = new T[newCap];

        for (size_t i = 0; i < mSize; i++)
            newArr[i] = mArr[i];

        delete[] mArr;
        mArr = newArr;
        mCapacity = newCap;
    }

    mArr[mSize++] = value;
}

void pop_back()
{
    if (mSize > 0) mSize--;
}

void resize(size_t n, T value = T())
{
    T* newArr = new T[n];
    size_t copySize = (mSize < n) ? mSize : n;

    for (size_t i = 0; i < copySize; i++)
        newArr[i] = mArr[i];

    for (size_t i = copySize; i < n; i++)
        newArr[i] = value;
}

```

```

        delete[] mArr;
        mArr = newArr;
        mSize = n;
        mCapacity = n;
    }

    void clear() { mSize = 0; }

    // 상태 조회
    size_t capacity() const { return mCapacity; }
    size_t size() const { return mSize; }
    bool empty() const { return mSize == 0; }

    // 비교
    bool operator==(const Vector& other) const
    {
        if (mSize != other.mSize)
            return false;

        for (size_t i = 0; i < mSize; i++)
            if (mArr[i] != other[i])
                return false;

        return true;
    }

    bool operator!=(const Vector& other) const
    {
        return !(*this == other);
    }

private:
    static constexpr size_t DEFAULT_CAP = 32;
    T* mArr;
    size_t mSize;
    size_t mCapacity;
};

```

```

}

int main()
{
    ya::Vector<int> vec;

    std::cout << "=== 기본 동작 테스트 ===" << std::endl;
    std::cout << "초기 크기: " << vec.size() << std::endl;
    std::cout << "초기 용량: " << vec.capacity() << std::endl;

    for (int i = 1; i <= 5; i++)
        vec.push_back(i);

    std::cout << "\n5개 요소 추가 후:" << std::endl;
    std::cout << "크기: " << vec.size() << std::endl;
    std::cout << "용량: " << vec.capacity() << std::endl;

    std::cout << "요소들: ";
    for (size_t i = 0; i < vec.size(); i++)
        std::cout << vec[i] << " ";
    std::cout << std::endl;

    std::cout << "반복자로 출력: ";
    for (auto it = vec.begin(); it != vec.end(); ++it)
        std::cout << *it << " ";
    std::cout << std::endl;

    std::cout << "\n=== resize 테스트 ===" << std::endl;
    vec.resize(15, 99);

    vec[0] = 2;
    vec[10] = 2;

    std::cout << "resize(15, 99) 후:" << std::endl;
    std::cout << "크기: " << vec.size() << std::endl;

    std::cout << "처음 15개 요소: ";

```

```

for (size_t i = 0; i < vec.size(); i++)
    std::cout << vec[i] << " ";
std::cout << std::endl;

std::cout << "\n=== 접근 함수 테스트 ===" << std::endl;
std::cout << "첫 번째 요소: " << vec.front() << std::endl;
std::cout << "마지막 요소: " << vec.back() << std::endl;

std::cout << "\n=== 복사 생성자 테스트 ===" << std::endl;
ya::Vector<int> vec2 = vec;

std::cout << "원본과 복사본 비교: " << (vec == vec2 ? "같음" : "다름") << std::endl;

vec2[0] = 100;
std::cout << "복사본 수정 후 비교: " << (vec == vec2 ? "같음" : "다름") << std::endl;

std::cout << "원본 첫 요소: " << vec[0] << std::endl;
std::cout << "복사본 첫 요소: " << vec2[0] << std::endl;

return 0;
}

```

커스텀 Vector 클래스 구현

이 페이지는 동적 배열을 직접 구현하면서 `std::vector`의 핵심 아이디어(메모리 관리, 크기/용량 분리, 복사 의미론)를 이해하는 것을 목표로 합니다. 실제 표준 라이브러리의 `std::vector`는 예외 안전성, 할당자(allocator), 이동 연산 최적화, 삽입/삭제 알고리즘 등 훨씬 더 많은 고려 사항이 있지만, 학습 관점에서는 아래 수준의 구현이 가장 중요합니다.

1) 클래스가 해결하는 문제

배열은 크기가 고정입니다. 반면, 벡터는 다음을 제공합니다.

- 필요할 때 자동으로 더 큰 메모리를 확보(재할당)해서 원소를 계속 추가할 수 있음
- 현재 원소 개수(`size`)와 확보된 메모리 크기(`capacity`)를 분리해서 성능을 확보

- 객체를 복사할 때 포인터만 복사되는 "얕은 복사" 문제를 피하기 위해 깊은 복사를 수행

2) 멤버 변수와 불변 조건(invariant)

```
static constexpr size_t DEFAULT_CAP = 32;
T*      mArr;          // 원소들을 담는 연속 메모리
size_t  mSize;         // 실제로 사용 중인 원소 개수
size_t  mCapacity;     // 확보된 메모리(원소를 담을 수 있는 총 칸 수)
```

다음 조건이 항상 성립하도록 유지하는 것이 핵심입니다.

- `0 <= mSize <= mCapacity`
- `mArr` 는 길이 `mCapacity` 만큼의 동적 배열을 가리킨다(또는 파괴 단계에서만 해제 직전)

3) 생성자/소멸자: RAII

- 생성자에서 자원을 얻고(메모리 할당)
- 소멸자에서 자원을 반납합니다(메모리 해제)

이 패턴을 RAII라고 하며, C++에서 자원 누수를 막는 가장 기본적인 구조입니다.

4) 복사 생성자/복사 대입: 깊은 복사

`Vector` 는 내부에 동적 메모리를 가지므로, 컴파일러가 자동으로 만들어주는 복사(멤버 단순 복사)를 쓰면 `mArr` 포인터가 그대로 복사되어 두 객체가 같은 배열을 공유하게 됩니다. 그러면 둘 중 하나가 파괴될 때 `delete[]` 가 수행되고, 다른 하나는 "이미 해제된 메모리"를 가리키게 되어 치명적인 버그(이중 해제, use-after-free)가 납니다.

따라서 복사 시에는 반드시 다음을 수행해야 합니다.

- 새 배열을 할당하고
- 원소들을 하나씩 복사한다

이것이 깊은 복사입니다.

5) push_back의 핵심: 재할당과 성장 전략

`push_back` 은 보통 다음 두 단계를 가집니다.

1. 공간이 남아 있으면 마지막에 값만 추가

2. 공간이 부족하면 더 큰 배열을 새로 만들고, 기존 원소를 복사한 뒤 교체

성장 전략을 "2배"로 잡는 이유는, 매 삽입마다 매번 재할당하지 않도록 해서 평균적으로 좋은 성능을 얻기 위함입니다(Amortized $O(1)$).

현재 구현은 `DEFAULT_CAP` 을 최소 용량으로 두고, 그 이후에는 2배로 늘립니다.

6) `resize`가 의미하는 것

- `resize(n)` 은 "논리적 크기"를 `n` 으로 맞춥니다.
- `n` 이 기존보다 크면 새로 늘어난 구간을 `value` 로 채웁니다.
- `n` 이 기존보다 작으면 뒤쪽 원소들을 버린 것으로 간주합니다.

주의: 표준 `std::vector` 의 `resize` 는 기존 capacity를 유지할 수 있지만, 여기 구현은 단순화를 위해 항상 정확히 `n` 크기의 새 배열을 할당합니다(즉, `capacity` 가 `n` 으로 맞춰짐).

7) 반복자(begin/end)

연속 메모리 기반 컨테이너의 가장 단순한 반복자는 "원소 포인터"입니다.

- `begin()` 은 첫 원소의 주소
- `end()` 는 마지막 원소 "다음" 주소

이 규칙을 따르면 `for (it = begin(); it != end(); ++it)` 형태가 자연스럽게 동작합니다.

8) 비교 연산자(==, !=)

- 크기가 다르면 즉시 false
- 크기가 같으면 모든 원소를 순서대로 비교

이 방식이 시퀀스 컨테이너의 가장 기본적인 동등 비교입니다.

9) 다음 확장 과제(표준에 더 가까워지기)

- `reserve(n)` : 재할당을 미리 해서 `push_back` 성능을 안정화
- `shrink_to_fit()` : 불필요한 capacity를 줄임
- `insert/erase` : 임의 위치 삽입/삭제
- 이동 생성자/이동 대입(`Vector(Vector&&)` , `operator=(Vector&&)`)

- 예외 안전성(strong/ basic guarantee), 그리고 `T`가 복사/이동 중 예외를 던질 때의 처리

위 기능들을 추가하면 실제 `std::vector`에 훨씬 가까운 설계 감각을 익힐 수 있습니다.